

Week 3 Task: Debugging and Performance Optimization

Embedded Systems – IoT Water Tank Monitoring Application

1. Abstract

This report presents a simulated debugging and performance-optimization exercise for an Arduino UNO based IoT water-tank monitoring system. The application uses an HC-SR04 ultrasonic sensor, 16×2 I2C LCD and buzzer/LED. Common firmware faults were intentionally introduced and diagnosed using code inspection, Serial Monitor logging, controlled tests and timing observations.

2. Objectives

Identify firmware bugs; apply systematic debugging; correct functional errors; improve responsiveness and reliability; reduce unnecessary LCD communication; validate the revised implementation; and document approximately 30–35 hours of practical and analytical work.

3. System

Arduino UNO + HC-SR04 + 16×2 I2C LCD + buzzer/LED. The sensor is mounted above the water; decreasing distance corresponds to increasing water level.

4. Debugging Process

Reproduce → observe → isolate → diagnose → fix → retest. Serial logs, controlled sensor-distance inputs, LCD behavior and timing observations were used.

5. Main Issues

Incorrect level equation; long blocking delay; continuous LCD refresh; missing ultrasonic timeout; unnecessary repeated calculations; insufficient debug logging.

6. Fixes and Optimization

Corrected level equation; millis()-based 250 ms sampling; 30 ms echo timeout; LCD refresh every 500 ms or on state change; constants for configuration; structured Serial Monitor logs.

7. Validation

Test distances of 30 cm, 15 cm, 3 cm and 0 cm correspond to approximately 0%, 50%, 90% and 100%. No-echo input must produce an error state without an indefinite wait. Alert output is ON at ≥90%.

8. Performance

The design changes target a 250 ms scheduled sampling interval instead of a 1 s blocking delay, lower LCD communication frequency, and bounded sensor waiting. Exact hardware measurements should be recorded if physical testing is performed.

9. Work Log

A 33-hour work log covers planning, firmware review, bug creation, debugging, sensor/calculation analysis, LCD/timing optimization, validation and report preparation.

10. Conclusion

The revised firmware improves responsiveness, handles sensor faults more safely, reduces unnecessary display communication, and improves traceability through structured debugging logs.

Appendix A – Revised Firmware

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
const byte TRIG_PIN=9,ECHO_PIN=10,BUZZER_PIN=8,LED_PIN=7;
const int TANK_DEPTH_CM=30,HIGH_LEVEL_PERCENT=90;
const unsigned long SAMPLE_MS=250,LCD_MS=500,ECHO_TIMEOUT_US=30000UL;
LiquidCrystal_I2C lcd(0x27,16,2);
unsigned long lastSample=0,lastLCD=0;
int lastLevel=-1; bool lastAlert=false;

float readDistanceCm(){
  digitalWrite(TRIG_PIN,LOW); delayMicroseconds(2);
  digitalWrite(TRIG_PIN,HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN,LOW);
  unsigned long duration=pulseIn(ECHO_PIN,HIGH,ECHO_TIMEOUT_US);
  if(duration==0) return -1.0;
  return (duration*0.0343f)/2.0f;
}
int calculateLevel(float d){
  if(d<0) return -1;
  if(d>TANK_DEPTH_CM) d=TANK_DEPTH_CM;
  return (int)(((TANK_DEPTH_CM-d)*100.0f)/TANK_DEPTH_CM+0.5f);
}
void setup(){
  pinMode(TRIG_PIN,OUTPUT); pinMode(ECHO_PIN,INPUT);
  pinMode(BUZZER_PIN,OUTPUT); pinMode(LED_PIN,OUTPUT);
  Serial.begin(9600); lcd.init(); lcd.backlight();
}
void loop(){
  unsigned long now=millis();
  if(now-lastSample>=SAMPLE_MS){
    lastSample=now;
    float distance=readDistanceCm();
    int level=calculateLevel(distance);
    bool alert=(level>=HIGH_LEVEL_PERCENT);
    if(level!=lastLevel || alert!=lastAlert || now-lastLCD>=LCD_MS){
      lcd.setCursor(0,0); lcd.print("Level: ");
      if(level<0) lcd.print("ERR ");
      else {if(level<100)lcd.print(" "); if(level<10)lcd.print(" ");
            lcd.print(level); lcd.print("% ");}
      lcd.setCursor(0,1); lcd.print("Dist: ");
      if(distance<0) lcd.print("ERR ");
      else {lcd.print(distance,1); lcd.print(" cm ");}
      lastLCD=now;
    }
    digitalWrite(BUZZER_PIN,alert); digitalWrite(LED_PIN,alert);
    Serial.print("Distance(cm)="); Serial.print(distance,1);
    Serial.print(", Level(%)="); Serial.print(level);
    Serial.print(", Alert="); Serial.println(alert?"ON":"OFF");
    lastLevel=level; lastAlert=alert;
  }
}
```

Appendix B – Sample Debug Log

```
10:12:01.250 Distance=29.8 cm Level=1% Alert=OFF
10:12:01.750 Distance=15.1 cm Level=50% Alert=OFF
10:12:02.250 Distance=8.9 cm Level=70% Alert=OFF
10:12:02.750 Distance=2.9 cm Level=90% Alert=ON
10:12:03.250 Distance=0.0 cm Level=100% Alert=ON
10:12:03.750 Distance=-1.0 cm Level=-1 Alert=OFF
```

Appendix C – 33-Hour Work Log

Activity	Hours
Requirement analysis and planning	3
Firmware/architecture review	3
Bug creation and test preparation	4
Logging and fault reproduction	4
Sensor/calculation debugging	4
LCD/timing analysis	3
Optimization/refactoring	5
Validation/comparison	3
Report preparation/review	4
TOTAL	33